

GPG : GNU Privacy Guard

Version du 15 septembre 2026, 16:52

Objectifs

Le but de cette séance est de vous faire mettre en pratique les concepts de clés publiques, clés privées et de signatures électroniques à travers la boîte à outils GPG.

1 Présentation de GPG

Le format OpenPGP est le standard ouvert issu de PGP et largement déployé en particulier dans les distributions linux à travers l'outil GnuPG. Deux principaux logiciels ont adopté le format OpenPGP : GnuPG qui est gratuit et PGP qui est payant. Notez que GnuPG est également disponible sous windows meme si dans le cadre de ce TP nous ne l'utiliserons que sur Linux. Toutes les manipulations qui seront présentées dans la suite sont donc reproductibles sous windows.

▷ Exercice 1 *Rappels et découverte de l'outil*

Pour répondre aux différentes questions, vous pouvez vous référer au cours pour les questions générales et à la documentation de l'outil pour les aspects plus techniques. La documentation s'obtient avec la commande `# man gpg` et un aperçu des commandes disponibles avec `# gpg --help`

1. GPG est-il utilisé pour la cryptographie symétrique ou asymétrique ?
2. Quels sont les algorithmes de chiffrement et de signature fournis par l'outil ?

▷ Exercice 2 *Génération de son couple de clés*

Avant toute utilisation, il est fortement recommandé de générer son couple de clés pour pouvoir chiffrer et authentifier ses messages.

`# gpg --full-generate-key` (ou en fonction de votre version de gpg : `# gpg --gen-key`)

Choisissez RSA et RSA dans la liste qui est proposée, puis une clé de 2048 qui n'expire jamais. Saisissez ensuite votre nom et votre adresse e-mail puis tapez une pass phrase. **Attention, vous devez mémoriser cette pass phrase ne tapez donc pas n'importe quoi !**

1. Quel est le rôle de cette passphrase ?
2. A quoi correspondent les champs `pub`, `sub` et `uid` ?

Exportez ensuite vos deux clés dans des fichiers de la forme :`nomPubKey.gpg` et `nomPrivKey.gpg` où vous remplacerez `nom` par votre nom.

```
# gpg --output nomPubKey.gpg --armor --export "Identifiant"
```

Cette commande permet d'exporter la clé publique.

- Quelle est l'instruction qui permet d'exporter la clé privée ?
- Que veut dire l'option `--armor` ?
- Une fois les deux clés stockées dans des fichiers, faites passer votre clé publique à l'un de vos camarades.
- A la réception d'une clé publique, stockez la dans votre trousseau de clés grâce à l'option `--import`
- Visualisez le contenu de votre trousseau de clés à l'aide de l'option `--list-key`
- Il est également possible de visualiser ses clés privées grâce à `--list-secret-key`

▷ Exercice 3 Génération du certificat de révocation

Il est de coutume lors de la création du couple de clé de générer un certificat de révocation en particulier si vous avez généré un couple n'ayant pas de date d'expiration. En effet, si vous oubliez votre pass phrase, le certificat de révocation sera votre seul moyen d'invalider votre couple de clés.

```
# gpg --output revoc-key.asc --gen-revoke "Identifiant"
```

▷ Exercice 4 Chiffrement d'un texte

- Commencez dans un premier temps par saisir un petit texte de votre choix dans un éditeur de texte et sauvez le message dans le fichier `message.txt`

```
# nano message.txt
```
- Chiffrez ensuite le message à l'aide de votre clé publique. La sélection de la clé se fait à l'aide de l'option `--recipient "Identifiant"` et le chiffrement grâce à l'option `--encrypt`
- Si tout c'est bien déroulé, un message `message.txt.gpg` a été créé. Vérifiez que le contenu du message est bien illisible.
- Supprimez votre message d'origine, puis essayez de déchiffrer le message à l'aide de votre clé privée.
- Une fois que tout fonctionne, répétez l'opération en chiffrant cette fois ci le message avec la clé publique d'un de vos camarades et envoyez lui le message.

▷ Exercice 5 Signature électronique

L'objectif est de signer un message numériquement et de le faire ensuite valider par votre camarade. Pour générer une signature, vous pouvez utiliser l'instruction suivante :

```
# gpg --default-key "Identifiant" --armor --detach-sign message.txt
```

- Quel est votre fichier de signature généré ? y-a-t-il une extension ?

- Transmettez le message en clair et la signature à votre camarade pour qu'il en vérifie l'intégrité.
- Quelle est la commande qui permet de vérifier une signature ?

Dans cette première version, la signature est détachée du fichier. Il est également possible de générer un texte en clair intégrant la signature grâce à l'option `--clearsign`

- Vérifiez l'intégrité du fichier signé puis modifiez le. Que se passe-t-il ?

▷ **Exercice 6** *Relation de confiance*

La relation de confiance dans GPG s'établit grâce au système de signature. Comme nous sommes dans un mode pair-à-pair sans autorité centrale, c'est les signatures successives des clés par différents utilisateurs qui renforce la confiance. Certains sites comme par exemple <http://pgp.mit.edu> servent d'annuaire de clés pgp publique et facilitent ainsi la diffusion des clés signées.

- Rendez vous sur le site <http://pgp.mit.edu> ou sur <https://keyserver.ubuntu.com/> et effectuez une recherche sur Olivier Blazy. Plusieurs clés devraient normalement apparaître.
- Regardez la première clé. Elle présente normalement une auto-signature. A quoi cela sert-il de signer sa propre clé ?
- Comparez maintenant cette clé avec la clé olivier.blazy@ens.fr.
- Regardez les clés des signataires.

Toutes ces signatures de clés permettent d'établir ce que l'on appelle un graphe de confiance (Web of Trust).

▷ **Exercice 7** *Vérification des clés*

Comme dans toutes architecture classiques basées sur des couples (clé publique, clé privée) la confiance que l'on accorde aux clés que l'on reçoit est très importante. Pour améliorer le niveau de confiance des clés, une solution consiste à signer la clé publique d'un correspondant de confiance à l'aide de sa propre clé.

Un outil comme **seahorse** permet de visualiser son trousseau de clé ainsi que le niveau de confiance que l'on accorde à chaque clé.

- Installez **seahorse** s'il n'est pas déjà présent sur votre distribution grâce à la commande

```
sudo apt-get update; sudo apt-get install seahorse
```
- Visualisez le niveau de confiance de chacune des clés (**ne modifiez rien !**)

La première étape consiste à s'assurer de l'authenticité de la clé. Une solution simple proposée par **gpg** est l'empreinte de la clé. En effet cette empreinte peut être échangée à travers un canal sécurisé comme le téléphone ou même une carte de visite.

- Visualisez l'empreinte de votre clé publique et comparez la avec la clé que vous avez envoyée à votre camarade grâce à l'option `--fingerprint`.
- Signez les différentes clés publiques que vous ont fait passer vos camarades.
- Échangez les clés entre vous et recommencez le processus.

- Établissez un graphe de confiance permettant d'améliorer le niveau de confiance de chacune de vos clés.
- Visualisez dans **seahorse** le niveau de confiance des clés signées. Est ce que cela a changé par rapport aux clés non signées ?

▷ **Exercice 8** *OpenPGP et les mails*

Lancez thunderbird (installez le si nécessaire à l'aide de la commande `# sudo apt-get install thunderbird`) et configurez votre compte de messagerie Unilim (Créer un compte->détection automatique->édition manuelle) en IMAP. Les détails des champs à remplir peuvent être retrouvés sur la page de la DSI¹.

Installez ensuite le package **enigmail** (`# sudo apt-get install enigmail`) puis redémarrez firefox. Vous avez maintenant normalement un onglet OpenPGP qui apparaît dans Thunderbird dans la barre de menu.

- En utilisant ce nouveau menu, envoyez un message chiffré sur votre boîte personnelle. Comment se présente-t-il ?
- Envoyez ensuite un message chiffré vers un voisin vous ayant fourni sa clé publique et vérifiez qu'il peut bien consulter votre message.

1. <https://support.unilim.fr/messagerie/messagerie-etudiante/messagerie-et-agenda-etudiants/>